

Midterm — Solutions

Software Engineering — UATX — Spring 2026

1 — SOLUTION

`engine.begin()` commits on success, rolls back on exception

A: 3. Inside the `try` block, the third `INSERT` happens on the same connection that runs the `SELECT COUNT(*)` — it sees its own uncommitted writes. The transaction has not yet committed, but the `SELECT` runs in the same transaction, so it counts the new row.

B: 2. *Why:* `engine.begin()` is a transaction context manager — it commits on normal exit and ROLLS BACK on exception. The `RuntimeError` propagated past the `with` block before COMMIT, so the third `INSERT` was undone. Only the two original rows remain.

C: `[(1, '00PS'), (2, 'second'), (3, 'fourth')]`. The third `with engine.begin()` block exited normally, so both the `UPDATE` (`id=1`'s body becomes `'00PS'`) and the `INSERT` (a fourth row at `id=3`) committed together.

Test file: isolation, ordering, weak assertions

Four real problems. Full credit for three.

1. **No `test_db` fixture — tests share whichever DB `db.engine` happens to point to.** On the first run, the file is empty, so `alice` gets created and the tests pass. On the second run, `alice` already exists, so `test_create_user_and_list_includes_them`'s POST returns 409 instead of 201 and the test fails for an unrelated reason. Fix: add the `test_db` fixture from class — swap `db.engine` to a fresh in-memory SQLite engine for each test, restore after.
2. **`test_get_user_returns_user_data` depends on `test_create_user_and_list_includes_them` running first** (so that `alice` exists). If pytest re-orders tests, runs them in parallel, or that test fails, this test breaks for the wrong reason. Fix: the test should arrange its own state — create `alice` inside the test before calling GET.
3. **`assert len(users) >= 1` is too weak.** It passes whether the response has 1 user or 10,000. Combined with the lack of isolation, it'd happily pass on a polluted DB. Fix: `assert exact length (== 1)` once isolation is in place, or assert a specific user is present and that's the only one expected.
4. **`test_create_user_and_list_includes_them` is doing two things at once.** AAA from class: one Arrange, one Act, one Assert. This test creates a user and tests the list endpoint — if the list assert fails, you don't know whether create or list is broken. Fix: split into `test_create_user_returns_201` and `test_list_users_includes_a_just-created_user`.

Bonus: `conftest.py` doesn't `yield` — if the fixture later grew teardown logic, it'd need `yield client`; not a defect today.

Notification preferences schema

1. Shape decision: separate table. The spec says 5–10 new event types per year; the wide approach (one column per (*event*, *channel*) on the users table) would mean a schema migration adding three columns each time, with a users table that grows sparse and ever-wider. A separate table absorbs new event types as rows instead of columns.

2. The SQL:

```
CREATE TABLE notification_preferences (
  user_id      INTEGER NOT NULL,
  event_type   TEXT     NOT NULL,
  channel      TEXT     NOT NULL,
  enabled      BOOLEAN NOT NULL DEFAULT TRUE,
  updated_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id, event_type, channel),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  CHECK (channel IN ('email', 'push', 'in_app'))
);
```

Defaults are seeded on user creation (one row per known event_type per channel; the one exception per the spec is ('mentioned_in_comment', 'push') with enabled = FALSE).

3. Real ambiguities in the spec (any one is acceptable; strong answers name the call they'd make):

- **What happens when a new event type ships?** Do existing users get rows auto-inserted with the default? Or is "no row" treated by app code as "default on"? *Call:* on each new event_type release, run a migration that inserts default rows for all existing users; that way the table is the source of truth and the app code doesn't have to special-case missing rows.
- **If a user has no row for some (event, channel) pair** — is that "default on" or "unset and we should treat it as on"? *Call:* guarantee every user has a row for every known pair (per above), so "no row" is impossible.
- **Does the "everything on, except mentioned_in_comment over push" default belong in the schema or in app code?** A column-level DEFAULT can't express "TRUE for most pairs, FALSE for one specific pair" — that has to live where the seeding happens. *Call:* seed defaults in the user-creation handler, not via column defaults.
- **What happens when a user is deleted?** *Call:* FOREIGN KEY ... ON DELETE CASCADE so their preferences go with them.
- **How is event_type validated?** Free-text TEXT, a CHECK constraint over a known set, or a separate event_types reference table? *Call:* TEXT for now; revisit if events grow

metadata.

Monster vs focused tests for checkout

Version B is the right answer. Reasons:

1. **Failure localization.** If a test in B fails, the test name tells you which behavior broke (`test_checkout_with_empty_cart_400s`). If A fails on the second checkout assertion, all you know is some step in `test_complete_checkout_flow` broke.
2. **Early-fail masking.** In A, if the first POST 500s, the rest of the assertions never run, so a single execution shows one bug. B exposes them all per run.
3. **Tests as documentation.** Reading B's test names tells you the API contract; A's `test_complete_checkout_flow` tells you nothing.
4. **Easier to extend.** Adding "checkout with bad payment token returns 402" in B is a new function. In A, it's editing a 20-line test and figuring out where to insert the assert.

Specific case A handles better (either of these earns full marks):

- **Cart-empties-after-checkout invariant.** A verifies it directly — right after the successful checkout, in the same test that confirmed the items were there. None of B's individual tests checks that side-effect. (B has `test_checkout_with_empty_cart_400s`, but that starts from a fresh cart, so it doesn't catch a regression where a successful checkout fails to clear the cart.)
- **Order total matches cart total.** A asserts `order["total_cents"] == 4500` after checkout, catching a regression where the order is created with the wrong price. B's `test_checkout_creates_paid_order` only checks `status == "paid"` and never asserts on the order's total.

What other tests would you add to B (any two or three of these earn full marks):

- **The cart-empties-after-checkout test** — the case A handles that B doesn't.
- **Adding an item with `qty=0` or `qty=-1`** — should be 400 / 422.
- **Adding an unknown SKU** — should be 404 or 400.
- **Checkout twice in a row from the same cart** — the second one should 400 because the cart is empty after the first.
- **Adding the same SKU twice** — should the qty stack into one line item, or get rejected? Tests both that the spec is decided AND that the implementation matches.
- **Checkout with a missing `payment_token`** — should be 400.

Picking A and defending it well ("for a release smoke test, the linear story is the point and a single failure halts the deploy") also earns full marks on question 1, and questions 2 and 3 still apply: B's failure-localization is the case A doesn't handle as well, and "what other tests would

you add to A" should be a list of unrelated cases (bad SKU, bad qty, etc.) since A already covers the happy-path E2E.

Test boundary for `charge_user`

1. Boundaries:

- **Database: in-memory test version.** Spin up a fresh `sqlite:///memory:` engine in a `test_db` fixture — the same pattern from class. Fast, isolated, exercises the real SQL strings (mocking the DB hides bugs in those strings).
- **Stripe: mock.** Real calls cost money, require network access, are slow, and can flake. Patch `stripe.PaymentIntent.create` with `unittest.mock` and assert on what it was called with — the "when to mock" lesson from 4.2.

2. Three tests:

```
def test_charge_raises_if_no_payment_method(test_db, mock_stripe):
    # no row in payment_methods for user_id=42
    with pytest.raises(ChargeError, match="no payment method"):
        charge_user(42, 5000, test_db)
    mock_stripe.PaymentIntent.create.assert_not_called()

def test_charge_calls_stripe_with_correct_amount(test_db, mock_stripe):
    test_db.execute(text(
        "INSERT INTO payment_methods (user_id, stripe_pm_id) "
        "VALUES (42, 'pm_x')"))
    mock_stripe.PaymentIntent.create.return_value = SimpleNamespace(
        id="ch_123", status="succeeded")
    charge_user(42, 5000, test_db)
    mock_stripe.PaymentIntent.create.assert_called_once_with(
        amount=5000, currency="usd",
        payment_method="pm_x", confirm=True,
    )

def test_charge_writes_audit_row_on_success(test_db, mock_stripe):
    test_db.execute(text(
        "INSERT INTO payment_methods (user_id, stripe_pm_id) "
        "VALUES (42, 'pm_x')"))
    mock_stripe.PaymentIntent.create.return_value = SimpleNamespace(
        id="ch_123", status="succeeded")
    charge_user(42, 5000, test_db)
    row = test_db.execute(text(
        "SELECT * FROM charges WHERE user_id = 42"))
    row.first()
```

```
assert row.stripe_id == "ch_123"  
assert row.amount_cents == 5000
```

3. Test you would NOT write:

- "Test that `stripe.PaymentIntent.create` actually charges the card." That's testing Stripe, not your code — their service has its own tests, and running it from your CI is slow, costs money, and turns a unit-of-behavior test into an integration test of someone else's API.
- OR: "Test that the `INSERT` writes to the correct file on disk." That's testing SQLite/SQLAlchemy, not your code.
- OR: a test that asserts BOTH that Stripe is called AND that the audit row is written, in the same test — that's two acts (4.1's AAA), and when it fails you don't know which behavior broke.